



CONFIG · SECRETS & ACCESS CONTROL

Book 02 of the series — stop baking config into images. ConfigMaps for plain settings, Secrets for sensitive data (and the base64 truth), RBAC for who may do what, ServiceAccounts for workload identity, SecurityContext & Pod Security Standards for hardening, and Requests/Limits + QoS for the resource contract.



ConfigMap

plain config



Secret

sensitive data



RBAC

who may do what



ServiceAccount

workload identity



SecurityContext

pod hardening



PSS

three levels



Requests/Limits

the budget



QoS

eviction order

PART I · CONFIG INJECTION

- ☐ **CH 01** ConfigMaps — What & Why p.3
- ☐ **CH 02** Creating & Mounting ConfigMaps p.4
- ☐ **CH 03** Hot-Reload — What Actually Updates p.5

PART II · SECRETS

- ☐ **CH 04** Secrets — vs ConfigMaps p.6
- ☐ **CH 05** Secret Types & Creation p.7
- ☐ **CH 06** Mounting Secrets + imagePullSecrets p.8
- ☐ **CH 07** Base64 ≠ Encryption — The Truth p.9

PART III · ACCESS CONTROL (RBAC)

- ☐ **CH 08** RBAC — The Big Idea p.10
- ☐ **CH 09** Roles & RoleBindings p.11
- ☐ **CH 10** ClusterRoles & When to Use Which p.12
- ☐ **CH 11** ServiceAccounts p.13

PART IV · HARDENING & RESOURCES

- ☐ **CH 12** SecurityContext p.14
- ☐ **CH 13** Pod Security Standards p.15
- ☐ **CH 14** Requests & Limits p.16
- ☐ **CH 15** QoS Classes p.17

PART V · LABS

- ☐ **LAB 1** ConfigMap: Prove Env ≠ Hot-Reload p.18
- ☐ **LAB 2** Secret: Encoded, Not Encrypted p.19
- ☐ **LAB 3** RBAC: Can-I as a ServiceAccount p.20
- ☐ **LAB 4** Trigger an OOMKill on Purpose p.21

PART VI · PROVE IT

- ☐ **QUIZ** Self-Check Quiz p.22
- ☐ **KEY** Answer Key + Scoring p.23
- ☐ **GLOSS** Glossary p.24
- ☐ **CHEAT** One-Page Cheat Sheet p.25
- ☐ **FINAL** Where to Next → Book 03 p.26



Assumes Book 01 complete. You already know Pods, ReplicaSets, Deployments, self-healing, and NodePort. This book adds the config layer (I–II), the permission layer (III), and the hardening/resource layer (IV) — then proves it all with labs.



The problem: bake `LOG_LEVEL=debug` into your image and every environment change = **rebuild, re-push, re-pull**. A ConfigMap decouples configuration **from** the image: one image, many configs.

❌ CONFIG BAKED INTO THE IMAGE

```
app:v1 + debug → build → push
app:v1 + info  → build → push
app:v1 + prod  → build → push
3 images, 3 pipelines, 3 chances to drift
```

✅ CONFIG INJECTED AT RUNTIME

```
app:v1 (built ONCE)
+ cfg-debug → dev cluster
+ cfg-staging → staging
+ cfg-prod → prod
same image everywhere — only the map differs
```

🔧 Create one — three ways

```
$ kubectl create configmap app-config --from-literal=LOG_LEVEL=debug
$ kubectl create configmap app-conf --from-file=app.conf
$ kubectl apply -f configmap.yaml # the declarative way (CH 02)
configmap/app-config created
```



One sentence: a ConfigMap is a named bag of key/value pairs (or files) that lives *inside a namespace* — injected into Pods as **env vars** or **files**. It's the Kubernetes take on 12-factor config: *config belongs to the environment, not the artifact*.

🔗 How do Pods actually consume one? ← [CH 02: Creating & Mounting](#)

↑ back to top



```

apiVersion: v1
kind: ConfigMap
metadata: { name: app-config }
data:
  LOG_LEVEL: "debug"           # simple key → env var
  app.conf: |                  # whole file as one key
    listen 8080
    workers 4

```

🔑 Two consumption methods — they behave differently

```

spec:
  containers:
    - name: app
      image: myapp:2.0
      env:                                # METHOD 1 · single keys as env vars
        - name: LOG_LEVEL
          valueFrom:
            configMapKeyRef: { name: app-config, key: LOG_LEVEL }
      envFrom:                            # ...or import EVERY key as env
        - configMapRef: { name: app-config }
      volumeMounts:                       # METHOD 2 · keys as FILES
        - { name: cfg, mountPath: /etc/app }
  volumes:
    - name: cfg
      configMap: { name: app-config } # each key becomes a file

```

	📦 As env vars	📁 As volume files
Shape	flat <code>KEY=value</code>	real files (<code>app.conf</code>), nested keys ok)
Update behavior	never — snapshot at container start	auto-updated by kubelet (~1 min, CH 03)
Best for	simple toggles, ports, feature flags	config files, templates, anything the app re-reads



Rule of thumb: env vars are a **snapshot**, volume mounts are a **live feed**. Choose by asking: *"does my app re-read its config file, or read the environment once at boot?"*

↑ back to top



The most-Googled ConfigMap question. The answer is a **2×2 matrix** — and one famous trap.

Consumed as...	ConfigMap changes → Pod sees...	Why
Env var	✗ Never	environment is fixed at container start (Linux process rules, not k8s')
Volume file	✓ Yes — eventually (~1 min)	kubelet syncs mounted files on its sync period
Volume via subPath	✗ Never	subPath mounts a <i>copy</i> ; the symlink chain is broken
Env + app restart	✓ Yes (on next start)	new container = fresh snapshot



"Eventually" means eventually: updates ride the kubelet sync period plus the API watch cache — **up to ~1 minute of delay**, in no guaranteed order. Apps that cache config at startup (Java, many frameworks) still see **nothing** until they re-read the file. The file changing ≠ your app noticing.

```
volumes:
- name: cfg
  configMap: { name: app-config, immutable: true } # freeze it entirely
```

`immutable: true` — rejects all updates (and can't be un-set without delete/recreate). Trade-off: **zero hot-reload** in exchange for protecting against accidental changes + better kubelet performance at scale.

Common production pattern: app watches its config file (or a sidecar triggers a reload endpoint) — k8s delivers the file, *your app* decides to care.



Exam-grade answer: *volume-mounted ConfigMaps auto-update (eventually, ~1 min); env vars and subPath mounts never update — a Pod restart is the only guaranteed way to re-read env config. Lab 1 proves it.*

[↑ back to top](#)



Same mechanism, different cargo. A Secret holds data you'd flinch to print in a `git log`: passwords, tokens, TLS keys, registry credentials.

Aspect	⚙️ ConfigMap	🔒 Secret
Cargo	plain config	sensitive data (passwords, certs, tokens)
Values on disk	plain text	base64-encoded (<code>data</code>) — CH 07 for what that does and doesn't mean
Value size	~1 MiB	~1 MiB
Consumed as	env / volume	env / volume — same two methods
Default visibility	anyone who can <code>get</code> it	gated by RBAC — reading secrets is its own permission (Part III)
Write convenience	<code>data</code>	<code>data</code> (pre-encoded) or <code>stringData</code> (plain — API encodes for you)



Do not skip this: base64 is **encoding, not encryption** — anyone can decode it with one command. By default, Secrets sit in etcd unencrypted. Real protection = etcd encryption-at-rest + tight RBAC (Part III). Full autopsy in CH 07.

```
$ kubectl create secret generic db-pass --from-literal=PASSWORD="s3cr3t"
secret/db-pass created
```



Mental model: ConfigMap = the whiteboard in the apartment (anyone may read). Secret = the safe (base64 = the box it ships in; RBAC = who has the key). Same delivery mechanism, different trust level.

[↑ back to top](#)



`type` is a label that tells Kubernetes (and tooling) what shape the keys should take. The default is `Opaque` — "just key/values".

Type	Required keys	Used by
<code>Opaque</code> default	anything you like	app passwords, API keys, generic data
<code>kubernetes.io/dockerconfigjson</code>	<code>.dockerconfigjson</code>	imagePullSecrets — private registry auth (CH 06)
<code>kubernetes.io/tls</code>	<code>tls.crt</code> + <code>tls.key</code>	Ingress TLS termination (Book 03)
<code>kubernetes.io/basic-auth</code>	<code>username</code> , <code>password</code>	basic-auth integrations
<code>kubernetes.io/service-account-token</code>	— (API-managed)	SA tokens — mostly automatic since v1.24 (CH 11)

Creation — imperative & declarative

```
$ kubectl create secret generic db-pass --from-literal=PASSWORD="s3cr3t"
$ kubectl create secret docker-registry regcred \
  --docker-server=registry.example.com --docker-username=ci \
  --docker-password=$TOKEN
$ kubectl create secret tls web-tls --cert=tls.crt --key=tls.key
```

```
apiVersion: v1
kind: Secret
metadata: { name: db-pass }
type: Opaque
stringData:
  PASSWORD: "s3cr3t"
# WRITE-only convenience — plain text in
# the API base64-encodes it into `data`
```



`stringData` never comes back out — `kubectl get secret -o yaml` always shows `data` with base64 values. Handy for readable YAML in Git pipelines (with the Git-secrets caveat of CH 07).



Remember the pair: `dockerconfigjson` → pulling images · `tls` → serving HTTPS. These two built-in types are the exam's favorites because they have **mandatory key names**.

↑ back to top



Same two doors as ConfigMaps — env vars (snapshot) and volumes (live feed) — plus a third, special-purpose door for the registry.

```
spec:
  containers:
  - name: app
    image: myapp:2.0
    env:
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef: { name: db-pass, key: PASSWORD }
    volumeMounts:
    - { name: sec, mountPath: /etc/secrets, readOnly: true }
  volumes:
  - name: sec
    secret: { secretName: db-pass } # keys → files, kubelet-synced
```



Prefer the volume door for sensitive data: env vars leak casually — they appear in `kubectl describe`, crash dumps, child processes (`/proc/<pid>/environ`), and some error reporters. A read-only file at a known path is easier to contain. (Hot-reload semantics from CH 03 apply identically to Secret volumes.)



imagePullSecrets — auth for pulling images

```
# one-time: create the registry credential (CH 05)
$ kubectl create secret docker-registry regcred ...
# then attach it to the ServiceAccount — every Pod in that
# namespace using this SA inherits it (recommended way)
$ kubectl patch serviceaccount default -p \
  '{"imagePullSecrets":[{"name":"regcred"}]}'
# or per-Pod in the spec:
spec:
  imagePullSecrets:
  - { name: regcred }
```



Three doors, one rule: env = snapshot · volume = live feed · `imagePullSecrets` = only for the **kubelet pulling images** (never appears inside your container). SA-level attachment wins over per-Pod because it covers every Pod the SA runs.

↑ back to top



The single most common Secrets misconception. Watch it die in one command:

```
$ kubectl get secret db-pass -o jsonpath='{.data.PASSWORD}'
czNjcjN0          # looks safe... it is not
$ echo czNjcjN0 | base64 -d
s3cr3t            # decoded by anyone, instantly
```



Base64 is a transport encoding — it makes binary-ish data YAML-safe. It's a locked box made of cardboard: the point is safe shipping, not safe keeping. **Anyone** with `get secrets` permission reads every "hidden" value.

🍌 The four real layers of Secret protection

```
Layer 1 • Encoding      base64  NOT protection — just transport
Layer 2 • Encryption at rest  EncryptionConfiguration on kube-apiserver
                             (aescbc / secretbox / KMS plugin) — encrypts in etcd
Layer 3 • RBAC          who may get/list secrets at all ← Part III
Layer 4 • Externalization External Secrets / Vault — cluster
                             never stores the real value long-term
```

❌ Never commit raw manifests with real values to Git `stringData` + sealed/external secret tooling, or SOPS-style encryption — use

✅ Check your cluster: etcd encryption-at-rest is **opt-in** — many dev clusters run without it



Exam tip: "Are Secrets encrypted?" — the honest answer is "**encoded by default; encrypted only if etcd encryption-at-rest is configured.**" Questions usually probe exactly this distinction.

↑ back to top



Authentication asks "**who are you?**" — authorization asks "**may you do that?**". RBAC is Kubernetes' answer to the second question, and it's one grammatical sentence:

WHO User · Group · ServiceAccount may **VERB** get · list · watch · create · update · patch · delete on **RESOURCE** pods · secrets · deployments ... in **SCOPE** namespace or cluster

THE FOUR OBJECTS

Role	rules, namespace
RoleBinding	attaches subjects → Role (ns)
ClusterRole	rules, cluster-wide
ClusterRoleBinding	subjects → ClusterRole (all)

DEFAULT POSTURE

Out of the box, RBAC grants regular users **nothing** — no Role = no access. Even the **default ServiceAccount** in each namespace has **zero API permissions** (a hardening choice since v1.22). Every capability is granted *explicitly*.

```
$ kubectl auth can-i delete pods -n dev
no # ← your first authorization question
```



`kubectl auth can-i <verb> <resource> [-n ns]` is your constant companion — and with `--as=...` it answers for *other* identities (CH 09, Lab 3). Admins live in this command.



Carry the sentence: *Role = the rules · Binding = who gets them · Scope = where they apply*. Everything in the next three chapters is this sentence with different nouns.

↑ back to top



A **Role** is a namespaced rulebook; a **RoleBinding** hands that rulebook to people. Worked example — a CI bot that may read pods and manage config, nothing else:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: ci-reader
  namespace: dev          # Roles live IN a namespace
rules:
- apiGroups: [""]          # "" = core group (pods, secrets...)
  resources: [pods, pods/log]
  verbs: [get, list, watch]
- apiGroups: [""]
  resources: [configmaps]
  verbs: [get, list, create, update, patch]
```

```
kind: RoleBinding
metadata: { name: ci-reader-bind, namespace: dev }
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: ci-reader          # the rulebook
subjects:                  # WHO receives it
- kind: ServiceAccount
  name: ci-bot
  namespace: dev
```

```
# verify — answer FOR the bot without being the bot:
$ kubectl auth can-i list pods -n dev \
  --as=system:serviceaccount:dev:ci-bot
yes
$ kubectl auth can-i delete pods -n dev \
  --as=system:serviceaccount:dev:ci-bot
no # not in the rulebook → denied
```



Bindings are the only on-switch. Creating a Role changes nothing by itself; access exists only when a Binding names a subject. And verbs are literal — `list` does not imply `delete`.

[↑ back to top](#)



A **ClusterRole** is a rulebook that isn't tied to a namespace. Pairing it with the right binding type decides its reach:

Pairing	Effective scope	Use when...
Role + RoleBinding	one namespace	team/app permissions (the 90% case)
ClusterRole + RoleBinding	one namespace per binding	reuse one rulebook in many namespaces — bind the <i>same</i> ClusterRole to <code>dev</code> , <code>qa</code> , <code>prod</code> teams separately
ClusterRole + ClusterRoleBinding	entire cluster	platform admins, controllers that watch everything
Role + ClusterRoleBinding	✗ impossible — a cluster-wide binding can't reference a namespaced Role	



The classic trap: some resources are **cluster-scoped** (Book 01 · CH 11): *Nodes*, *PersistentVolumes*, *Namespaces*, *ClusterRoles*. They **cannot** be governed by a namespaced Role — you must use a **ClusterRole**, even if you only want narrow access to, say, *Nodes*.

```
# "let the monitoring bot READ nodes" — needs ClusterRole:
kind: ClusterRole
metadata: { name: node-reader } # note: no namespace field
rules:
- apiGroups: ["" ]
  resources: [nodes]
  verbs: [get, list, watch]
```



Why ClusterRole + RoleBinding is the pro move: one canonical rulebook (audited once), namespace-scoped blast radius per binding. ClusterRoleBinding is a big hammer — reserve it for identities that truly need the whole cluster.



Decision in one line: start with **Role + RoleBinding**; graduate to **ClusterRole + RoleBinding** for reuse; **ClusterRoleBinding** only for genuinely cluster-wide identities — and remember cluster-scoped resources *force* a ClusterRole.

[↑ back to top](#)



Users are for **people**; a **ServiceAccount (SA)** is an identity for **workloads** — the "staff badge" your Pod wears when it talks to the API Server.

🔑 Every namespace ships a `default` SA — Pods without an explicit `serviceAccountName` wear it

📁 A **projected** mounted `/var/run/secrets/kubernetes.io/serviceaccount` — that's the credential the app's Pod's SA **token** at `k8s client` uses (auto-discovered in-cluster) gets a

🚫 Modern RBAC gives the default SA **no permissions** — a Pod that doesn't call the API should opt out:

```
spec:
  serviceAccountName: ci-bot      # wear THIS badge
  automountServiceAccountToken: false # ...or wear none
```

🔗 The full chain — badge → rulebook

```
Pod wears ServiceAccount ci-bot
├──
├── RoleBinding grants ci-reader Role (CH 09)
│   ├──
│   └── result: list pods ✓ · delete pods ✗ in ns dev only
```

```
$ kubectl create serviceaccount ci-bot -n dev
$ kubectl set serviceaccount deploy/web ci-bot -n dev
$ kubectl auth can-i get secrets -n dev \
  --as=system:serviceaccount:dev:ci-bot
no # ← this is what a stolen Pod token can do. audit it.
```



The Pod-token threat model: anyone who can `exec` into your Pod can read its token and **become** that SA. Scope each SA's Role to the minimum (Lab 3 audits exactly this) — never run untrusted workloads under an admin-bound SA.



One sentence: *ServiceAccount = workload identity; RoleBinding = its permissions; the mounted token = its key. RBAC + SAs together answer "what damage can this Pod do?"*

↑ back to top

RBAC governs your Pod's *API* permissions; the SecurityContext governs its *Linux* permissions. Two levels apply:

Level	Fields	Notes
Pod-level	<code>spec.securityContext</code> <code>runAsUser</code> · <code>runAsGroup</code> · <code>runAsNonRoot</code> · <code>fsGroup</code>	defaults inherited by every container
Container-level	<code>containers[].securityContext</code> all of the above + <code>readOnlyRootFilesystem</code> · <code>allowPrivilegeEscalation</code> · <code>capabilities</code> · <code>privileged</code>	overrides pod-level per container

```
spec:
  securityContext:           # POD level
    runAsNonRoot: true      # refuse to start as uid 0
    runAsUser: 1000
    fsGroup: 2000           # group ownership for volumes
  containers:
  - name: app
    image: myapp:2.0
    securityContext:        # CONTAINER level
      readOnlyRootFilesystem: true # tamper-proof image FS
      allowPrivilegeEscalation: false
      capabilities:
        drop: ["ALL"]        # shed Linux superpowers
```

 `readOnlyRootFilesystem` → writes must go to mounted `emptyDir` volumes — a common first hardening step

 `privileged: true` ≈ root on the node **never** for untrusted workloads (PSS · CH 13 forbids it outright at Baseline+)



Remember the pairing: RBAC = may it call the API? · SecurityContext = what can it do to the machine?

Book 01's CH 07 taint story protected the node from scheduling; this protects the node from the container.

[↑ back to top](#)



Checking every Pod's SecurityContext by hand doesn't scale. **Pod Security Standards (PSS)** define three ready-made security levels, enforced cluster-wide by namespace labels.

**Privileged**

no restrictions — trusted, cluster-infrastructure workloads only

**Baseline**

blocks the obviously dangerous (hostNetwork, privileged:true) while allowing common apps

**Restricted**

current hardening best practice: runAsNonRoot, drop ALL capabilities, no privilege escalation

```
# enforce on a namespace — three modes per level:
$ kubectl label ns prod \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/audit=restricted \
  pod-security.kubernetes.io/warn=restricted
```

enforce — rejecting non-compliant Pods · **audit** — log violations · **warn** — message the user

Migration pattern: run `warn+audit` first, fix offenders, then switch on `enforce`



Version note: the old **PodSecurityPolicy (PSP)** was removed in Kubernetes **v1.25**. If a tutorial teaches PSP, it's stale — the modern mechanism is **PSS via Pod Security Admission** (the labels above).



One line: PSS = cluster policy, three levels (Privileged → Baseline → Restricted), applied per-namespace with enforce/audit/warn labels. "Restricted" ≈ your CH 12 SecurityContext, made mandatory.

[↑ back to top](#)

**REQUESTS — the promise**

What the container is **guaranteed**. The **Scheduler** places Pods only on Nodes with enough *requestable* capacity (Book 01 · CH 06's "Pending" mystery starts here).

LIMITS — the ceiling

The maximum the runtime **enforces**. What happens at the ceiling **differs radically by resource type** — the table below is the exam's favorite nuance.

```
containers:
- name: app
  image: myapp:2.0
  resources:
    requests: { cpu: "250m", memory: "256Mi" }
    limits:   { cpu: "500m", memory: "512Mi" }
```

Resource	Exceeding the limit →	Feels like
CPU (compressible)	Throttled — process slowed via CFS quota	sluggish app, elevated latency, no restart
Memory (incompressible)	OOMKilled — SIGKILL, exit code 137	container dies; RestartPolicy kicks in → often CrashLoopBackOff

```
$ kubectl describe pod oom-demo | grep -A3 Last State
Last State:    Terminated
Reason:       OOMKilled
Exit Code:    137  # = 128 + 9 (SIGKILL)
```



The overcommit trap: total requests < node capacity < total limits is legal and common — the cluster *borrows* headroom. When a node runs out of *real* memory, the kube evicts by QoS class — which is exactly CH 15.



Memory hook: CPU over limit = **tickets get slower** · memory over limit = **you're escorted out (137)**. CPU can be shared; memory cannot be un-consumed.

↑ back to top



Every Pod is silently stamped with a Quality-of-Security— sorry, Quality-of-**Service** class, derived **purely from its requests/limits**. It decides who gets evicted first when a node runs dry.

Guaranteed every container: `requests == limits` for **both** cpu AND memory
Burstable at least ONE requests/limits field set (anything else)
BestEffort no requests, no limits – anything

Class	Derivation	Eviction order under node pressure
Guaranteed	requests == limits (all containers)	last — killed only after others
Burstable	some resources specified	middle — those exceeding requests go first
BestEffort	nothing specified	first — zero promises, zero protection

```
$ kubectl get pod web -o jsonpath='{.status.qosClass}'
Burstable
```



Why this matters operationally: your database should be **Guaranteed** (it's the thing you least want killed); dev/experiment Pods can ride **BestEffort** as the cluster's shock absorber. QoS is how Kubernetes turns your YAML honesty into survival priority.



The chain, end to end: requests → where the Scheduler may place you · limits → what happens when you misbehave · QoS → who survives when the node itself starves. Part V now proves all of it hands-on.

[↑ back to top](#)



STEP 1 · One ConfigMap, both consumption methods

```
$ kubectl create configmap demo --from-literal=GREETING=hello
$ kubectl run cfg-demo --image=busybox --restart=Never \
  --overrides='{ "spec": { "containers": [ { "name": "c", "image": "busybox",
    "command": [ "sh", "-c", "sleep 9999" ],
    "env": [ { "name": "GREETING", "valueFrom": { "configMapKeyRef": { "name": "demo", "key": "GREETING" } } },
    "volumeMounts": [ { "name": "cfg", "mountPath": "/etc/cfg" } ] },
    "volumes": [ { "name": "cfg", "configMap": { "name": "demo" } } ] } ] } }
```

🕒 TAB 1 · OBSERVE

```
$ kubectl exec cfg-demo -- printenv GREETING
hello
$ kubectl exec cfg-demo -- cat /etc/cfg/GREETING
hello
```

⚙️ TAB 2 · CHANGE IT

```
$ kubectl patch cm demo -p \
  '{"data":{"GREETING":"howdy"}}'
configmap/demo patched
```

STEP 2 · Wait ~1 min, re-check

🕒 ENV VAR — the snapshot

```
$ kubectl exec cfg-demo -- printenv GREETING
hello # ← UNCHANGED, forever
```

🕒 VOLUME FILE — the live feed

```
$ kubectl exec cfg-demo -- cat /etc/cfg/GREETING
howdy # ← updated by kubelet (~1 min)
```

Proved: the matrix from CH 03 is real — env is a boot-time snapshot; volume mounts are kubelet-synced. **Teardown:**

```
kubectl delete pod cfg-demo && kubectl delete cm demo .
```

[↑ back to top](#)



STEP 1 · Create, mount, go inside

```
$ kubectl create secret generic lab-sec --from-literal=PASSWORD="hunter2"
$ kubectl run sec-demo --image=busybox --restart=Never \
  --overrides='{ "spec": { "containers": [ { "name": "c", "image": "busybox",
    "command": [ "sh", "-c", "sleep 9999" ],
    "volumeMounts": [ { "name": "s", "mountPath": "/etc/sec", "readOnly": true } ] },
    "volumes": [ { "name": "s", "secret": { "secretName": "lab-sec" } } ] } } }
```

STEP 2 · Read the "hidden" value — three levels of nakedness

```
# inside the pod — the file is ALREADY DECODED for the app:
$ kubectl exec sec-demo -- cat /etc/sec/PASSWORD
hunter2
# from the API — only base64 stands in the way:
$ kubectl get secret lab-sec -o jsonpath='{.data.PASSWORD}'
aHVudGVyMg==
$ echo aHVudGVyMg== | base64 -d # → hunter2. instantly.
```

Proved: CH 07's claim — base64 protects nothing; the kubelet even hands your app the decoded value. Protection lives in **who may run** `get secret` (RBAC) and etcd encryption-at-rest. **Teardown:** `kubectl delete pod sec-demo` && `kubectl delete secret lab-sec`.



Interview one-liner born from this lab: "Secrets protect against *casual exposure* and unauthenticated reads — not against anyone holding `get secrets` RBAC."

[↑ back to top](#)



STEP 1 · Build the chain: SA → Role → Binding

```
$ kubectl create ns rbac-lab
$ kubectl create serviceaccount bot -n rbac-lab
$ kubectl create role podlist --verb=get,list --resource=pods -n rbac-lab
$ kubectl create rolebinding bot-bind --role=podlist \
  --serviceaccount=rbac-lab:bot -n rbac-lab
```

STEP 2 · Interrogate the bot's permissions

```
$ K="--as=system:serviceaccount:rbac-lab:bot -n rbac-lab"
$ kubectl auth can-i list pods $K # in the rulebook
yes
$ kubectl auth can-i delete pods $K # verbs are literal!
no
$ kubectl auth can-i list secrets $K # resources too
no
$ kubectl auth can-i list pods -n default $K # scope is namespaced!
no
```

STEP 3 · What we proved

- ✓ The bot may **list pods** — exactly one namespace, exactly the granted verbs
- ✗ Delete? Secrets? Other namespaces? All **no** — default posture is deny; the Role's scope is absolute
- 🔍 `--as` lets you **audit any identity without their credentials** — the core of RBAC troubleshooting

Teardown: `kubectl delete ns rbac-lab` — the cascade (Book 01 · Lab 3) removes the whole chain.

[↑ back to top](#)

STEP 1 · Deploy a Pod with a tiny memory limit and a big appetite

```
$ kubectl set image deploy/oom-demo oom=polinux/stress \
-n default 2>/dev/null || kubectl create deploy oom-demo --image=polinux/stress \
-- stress --vm 1 --vm-bytes 200M --timeout 300s
$ kubectl set resources deploy/oom-demo --limits=memory=50Mi --requests=memory=30Mi
$ kubectl get pods -w
```

TAB 1 · WATCH

NAME	STATUS	RESTARTS
oom-demo-aaaaa	Running	0
oom-demo-aaaaa	Error	1 # OOM!
oom-demo-aaaaa	Running	1
oom-demo-aaaaa	Error	2 # again...

TAB 2 · AUTOPSY

```
$ kubectl describe pod -l app=oom-demo
Last State: Terminated
Reason: OOMKilled
Exit: 137
# 128+9 = SIGKILL by the
# kernel's OOM killer
```

STEP 2 · What we proved

- ✓ Memory limit exceeded → **OOMKilled (137)** , not throttled — CH 14's incompressible-resource rule
- ✓ The Deployment's reconciliation (Book 01 · CH 06) restarts it → it OOMs again → **CrashLoopBackOff** signature the classic
- 💡 Real-world translation: *app that crash-loops right after startup + exit 137 = raise the limit or fix the leak*

Teardown: `kubectl delete deploy oom-demo` . (Then check its QoS: it had requests≠limits → **Burstable** — CH 15 in the flesh.)

[↑ back to top](#)



Q1. A ConfigMap is consumed by a Pod both as an env var and as a volume file. You update the ConfigMap. What does the running Pod see?

- A. Both update within ~1 min B. Env unchanged; the volume file updates eventually C. Env updates; file unchanged D. Neither — a restart is always required

Q2. A colleague says Secrets are "encrypted by Kubernetes". The accurate response?

- A. True — always encrypted in etcd B. True, but only for Opaque type C. False — values are base64-*encoded*; encryption-at-rest must be explicitly enabled D. False — Secrets are stored in plain text in the API Server's memory

Q3. Which Secret type must contain the key `.dockerconfigjson` ?

- A. Opaque B. kubernetes.io/tls C. kubernetes.io/dockerconfigjson D. basic-auth

Q4. By default, which identity may read Secrets in namespace `dev` ?

- A. Any authenticated user B. The default ServiceAccount C. Cluster admins only, forever D. Nobody beyond cluster-admin-tier until an RBAC Role/Binding grants it

Q5. `kubectl auth can-i get pods --as=system:serviceaccount:dev:bot` answers...

- A. Can the bot authenticate? B. May *bot* perform *get pods*? (authorization, impersonated) C. Do you have permission? D. Does the Role exist?

Q6. A RoleBinding may reference which rulebook(s)?

- A. Role only B. ClusterRole only C. Role or ClusterRole D. Neither — bindings only reference users

Q7. A Pod is classed **Guaranteed** when...

- A. It runs on 3+ nodes B. limits only are set C. Every container sets requests == limits for both CPU and memory D. It has priorityClassName set

Q8. A container consistently exceeds its memory limit. Result?

- A. CPU throttling B. Pod rescheduled C. Limit auto-raised D. OOMKilled — exit code 137

Q9. `readOnlyRootFilesystem` is set at which level?

- A. Namespace B. Role C. Container securityContext D. ServiceAccount

Q10. Which PSS level requires `runAsNonRoot`, dropping ALL capabilities, and disabling privilege escalation?

- A. Privileged B. Restricted C. Baseline D. PodSecurityPolicy

[↑ back to top](#)



Q	Ans	Why — one line
1	B	Env = boot snapshot, never updates; volume files kubelet-synced ~1 min — CH 02/03
2	C	Base64 is transport encoding; etcd encryption-at-rest is opt-in — CH 07
3	C	dockerconfigjson type mandates the <code>.dockerconfigjson</code> key — CH 05/06
4	D	Default posture is deny; even the default SA has zero permissions — CH 08/11
5	B	can-i + <code>--as</code> = authorization check for an impersonated identity — CH 08/09
6	C	RoleBinding may bind a ClusterRole, scoping it to its namespace — CH 10
7	C	Guaranteed = requests==limits, cpu AND memory, every container — CH 15
8	D	Memory is incompressible → OOMKill, 128+9=137 — CH 14, Lab 4
9	C	readOnlyRootFilesystem is container-level SecurityContext — CH 12
10	B	Restricted = hardening best practice; PSP is removed (v1.25) — CH 13

**9–10**

Ready for Book 03

**6–8**Re-read the missed chapters' 
boxes**<6**

Re-run Labs 1–4, then retake



Missed Q2 or Q4? Those are the two real-world security gaps — "encoded ≠ encrypted" and "default deny".
Missed Q6? The ClusterRole+RoleBinding combo is the most misremembered fact in RBAC.

[↑ back to top](#)



ConfigMap — namespaced key/value (or file) store for plain config; consumed as env vars or volume files.

Hot-reload — volume-mounted ConfigMaps/Secrets update automatically (~1 min, eventual); env vars and subPath mounts never do.

Secret — ConfigMap's sensitive sibling; base64-encoded values, RBAC-gated reads, same two consumption methods.

Opaque — default Secret type: arbitrary key/value pairs.

imagePullSecrets — registry credentials used by the kubelet to pull private images; best attached at ServiceAccount level.

Base64 — transport encoding, not encryption. Decodable by anyone in one command.

Encryption at rest — opt-in kube-apiserver EncryptionConfiguration that encrypts Secrets inside etcd.

Authentication vs Authorization — who are you vs may you do that. RBAC answers the second.

Role / ClusterRole — namespaced / cluster-level rulebooks: verbs on resources.

RoleBinding / ClusterRoleBinding — attach subjects to rulebooks; a RoleBinding may bind a ClusterRole but scope it to its namespace.

Subject — User, Group, or ServiceAccount named by a Binding.

ServiceAccount — workload identity; its projected token (mounted in the Pod) is the credential Pods use against the API.

SecurityContext — Linux-level hardening: runAsNonRoot, readOnlyRootFilesystem, capabilities, allowPrivilegeEscalation (pod- and container-level).

Pod Security Standards — Privileged / Baseline / Restricted levels enforced per-namespace via pod-security.kubernetes.io labels (successor to removed PSP).

Requests / Limits — guaranteed capacity for scheduling / enforced ceiling (CPU throttles; memory OOMKills with exit 137).

QoS Class — Guaranteed (req==lim) · Burstable (partial) · BestEffort (none) — determines eviction order under node pressure.

[↑ back to top](#)



CONFIG & SECRETS — COMMANDS

```
$ kubectl create configmap cm --from-literal=K=V --from-file=f.conf
$ kubectl create secret generic s --from-literal=K=V
$ kubectl create secret docker-registry reg --docker-server=... --docker-username=...
$ kubectl create secret tls t --cert=crt --key=key
$ kubectl get secret s -o jsonpath='{.data.K}' | base64 -d
$ kubectl patch sa default -p '{"imagePullSecrets":[{"name":"reg"}]}'
```

RBAC — COMMANDS

```
$ kubectl create role r --verb=...
$ kubectl create rolebinding r --role=r --subject=...
$ kubectl create clusterrole cr --verb=...
$ kubectl auth can-i delete pods
$ kubectl describe role/role
```

HOT-RELOAD MATRIX

Consumed as	Updates?
env var	never
volume file	yes (~1 min)
subPath mount	never

QOS + ENFORCEMENT

```
Guaranteed: requests==limits (cpu+mem, all)
Burstable: partial · BestEffort: none
eviction: BestEffort → Burst>req → Guaranteed
mem over limit → OOMKilled 137
cpu over limit → throttled
$ kubectl get pod P -o jsonpath='{.status.qosClass}'
```

FACTS WORTH A JOB OFFER

ConfigMap = whiteboard, Secret = safe — same delivery, different trust · both ~1 MiB, namespaced

Base64 ≠ encryption — real layers: etcd encryption-at-rest + RBAC (+ external secret managers)

Default deny: no Role = no access; default SA has zero permissions; audit with `can-i --as`

RoleBinding may bind a ClusterRole (namespaced reach) · cluster-scoped resources force ClusterRoles

SecurityContext = Linux perms; PSS (Privileged/Baseline/Restricted) = namespace-enforced policy; PSP is gone (v1.25)

Requests → scheduling · Limits → CPU throttle / mem OOMKill · QoS → eviction order

[↑ back to top](#)



Book 01 gave you the machine. This book gave it **manners** — config, secrets, permissions, and a resource contract.

POD — fully dressed now:

- | **ConfigMap** plain config → env / files (hot-reload rules known)
- | **Secret** sensitive data — encoded ≠ encrypted, RBAC-gated
- | **ServiceAccount** + Role/RoleBinding = exactly what it may do
- | **SecurityContext** runAsNonRoot · roFS · caps (PSS-enforced)
- | **requests/limits** → scheduler placement, throttle/OOM, QoS class

✓ You can now...

Decouple config from images and pick env-vs-volume *knowing* the hot-reload consequences

Dismantle the "Secrets are encrypted" myth and name the four real protection layers

Write Role + RoleBinding, reuse a ClusterRole per-namespace, and audit anyone with `can-i --as`

Harden a Pod (SecurityContext) and enforce it fleet-wide with PSS namespace labels

Predict throttle-vs-OOMKill, and read a 137 exit code like a native speaker

Trigger — and diagnose — every failure mode in Labs 1–4 on demand



Where to next — Book 03 · Services & Networking Deep Dive: LoadBalancer, Ingress & TLS termination, NetworkPolicy, CoreDNS internals, and the Gateway API. Your Pods can now be configured, secured, and budgeted — Book 03 teaches the world how to *reach* them.

— End of Book 02 · —